

Multilane macros

First of all, a judicious decision has to be taken when a multilane activity needs to be enclosed in a macro or in a function. If unsure, use the keyword 'inline' and let the compiler make the decision.

Now, once it's been decided to write a multilane macro, it is preferable to enclose it in a 'do{}while(0)'. There's no semicolon after it. The semicolon is supposed to be given at the time of macro call.

This ensures that we can convert the macro into a function at any point in time, and we need not browse through the entire code or keep adding semicolons at a later point of time.

For example:

```
#define INIT(x,y,z) do{ x=0; \
if(x) y=0; \
else y=1}while(0)
```

The reasons for this *do-while(0)* business are:

- 1) The macro becomes a single logical statement for the compiler, and hence can be safely invoked as:
- ```
if (condition)
INIT(p,q,r);
else
/* something else */
```
- 2) The macro is guaranteed to execute once.
  - 3) The compiler almost always optimises out the *do-while* and hence there's no performance overhead.
  - 4) Incomplete statements in the macro throw compilation errors.

## Putting the constant on the left when comparing with a constant

Consider the following piece of C code:

```
define DELIMITING_CHARACTER '$'

/* somewhere down.....*/
char&tmp_char;
while(1)
{
 tmp_char = getchar();
 if(tmp_char == DELIMITING_CHARACTER)
 break;
 else
 putc(tmp_char);
}
```

This works perfectly. Now, what happens if you or a colleague mistakenly takes out a '=' sign from the 'if' statement in a hurry? The compiler will happily compile the code and the *while(1)* will keep

on executing endlessly.

A simple solution is to write the comparison in the following way:

```
tmp_char = getchar();
if(DELIMITING_CHARACTER == tmp_char)
 break;
```

Now even if the '=' becomes '=', the compiler will straight away throw a compilation error and attract your attention. Hence, when comparing with a constant, the constant should always be placed to the left side of comparison.

## Writing explicit parenthesis

An amateur programmer often tries to sound smart by dropping the parenthesis just because C allows it. But consider the following piece of code:

```
if(condition_outer)
 if(condition_inner)
 statement_1;
else
 statement_2;
```

In C, every 'else' branch belongs to the innermost possible 'if' statement, which in this example is statement\_2, and will be executed if, and only if, the condition\_inner is false. This is often not what the programmer expected.

Hence, it is suggested that programmers should use the explicit parenthesis at all the places:

```
if(condition_outer)
{
 if(condition_inner)
 (statement_1;)
 else
 (statement_2;)
}
```

...OR:

```
if(condition_outer)
{
 if(condition_inner)
 {
 statement_1;
 }
}
else
{
 statement_2;
}
```

...whichever logic is correct. The use of explicit parenthesis makes the code more readable and easy to understand.